

cloud quality also need to repair the holes caused by the object occlusion. Holes repairing algorithms based on triangular meshes could be divided into two categories: surface-based methods and volume-based ones [16]. Generally, surface-based methods detect, repair, and refine the holes on the given triangular mesh [17, 18]. Volume-based methods convert the given mesh into a function with signed distance on the volume mesh to fill the holes and then extract the complete grid from the zero-order set of the distance function [19–21]. The holes repairing algorithm proposed by Leong et al. [22] directly connect the boundary of the holes, but the repair is not effective since no new triangle surfaces are added. Liu Zhenghong and Yun [23] extracted the boundary of the holes and determined the direction of contraction by calculating the direction of the triangular surface's normal vector related to the boundary point. In this way, a complete triangular patch is iteratively generated, but the number of additional points could not be controlled. The point cloud quality optimization and enhancement algorithm designed in this paper could effectively remove the outliers and confounding points while maintaining the sharp features of the point cloud. At the same time, by setting the priority value, the number of new points could be controlled when repairing holes.

2. Theoretical Derivation

The implementation steps of the proposed method are as follows:

- (1) Establishing a self-adaptive octree to rasterize the point cloud and setting the thresholds for defining the outliers to delete these points
- (2) After removing the outliers, projecting the point cloud onto the local least squares fitting plane to remove the confounding points
- (3) Establishing a priority value and inserting points at the positions with higher priority values until the hole filling which is completed

2.1. Outlier Removal. Outliers are usually disorganized, far away from the main point cloud, sparse, and geometrically discontinuous with inconsistent local point density. In contrast, the main point cloud is relatively concentrated and dense. The statistical filtering algorithm is based on the characteristics that the distance between the outlier points and the neighboring points is considerable while the distance between the main points and the neighboring points is small. And the statistical analysis towards the neighborhood of each point is used for removing the outliers [24]. In this paper, density is introduced based on the statistical filtering algorithm. First, an octree is established to take the minimum enclosing cube of the point cloud as the root node, and the cube is divided into eight subcubes with equal size. If a subcube contains points, it should continue to be divided into eight cubes with equal size until each point in the point cloud obtains a unique index coordinate. Then, the density of the point cloud in each small cube is calculated, and usu-

ally, the density of the subcube containing outliers is relatively small.

The specific implementation steps are as follows:

- (1) Establishing a self-adaptive octree to rasterize the point cloud and calculating the density of the point cloud ρ_n in each grid
- (2) For any point $p_i \in P$ of the point cloud in each grid, searching for the k -neighborhood point $p_j (j \in k)$ of p_i and calculating the distance between p_i and p_j
- (3) Defining the probability of a point being an outlier as

$$\delta = \frac{\sum_{j=1}^k \|p_i - p_j\|}{k\rho_n} \quad (1)$$

- (4) Establishing a threshold. If δ is greater than the threshold, p_i is considered an outlier and is deleted

2.2. Confounding Point Removal. Confounding points cannot be easily distinguished from the real point cloud. It is difficult for traditional algorithms to achieve a balance between removing the confounding points and maintaining the detailed and sharp features of the point cloud. In this paper, we use the least square to fit the local plane and obtain the normal vector of the local plane. The mixed points are projected onto the local fitting plane, which not only improves the quality of the point cloud but also maintains the sharp features.

The least squares method is used for fitting the local plane and obtaining the normal vector. First, we fit a local plane with a set of points composed of a point $p_i \in P$ and its k -neighborhood points. Suppose the local fitting plane is $Z = Ax + By + C$, and then, the formula of the deviation quadratic sum is as follows:

$$d = \sum_{i=1}^{k+1} [(Ax_i + B_i + C) - z_i]^2. \quad (2)$$

It can also be written in matrix form as follows:

$$d = (UB - l)^2, \quad (3)$$

where

$$U = \begin{pmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & 1 \end{pmatrix},$$

$$B = \begin{pmatrix} A \\ B \\ C \end{pmatrix},$$

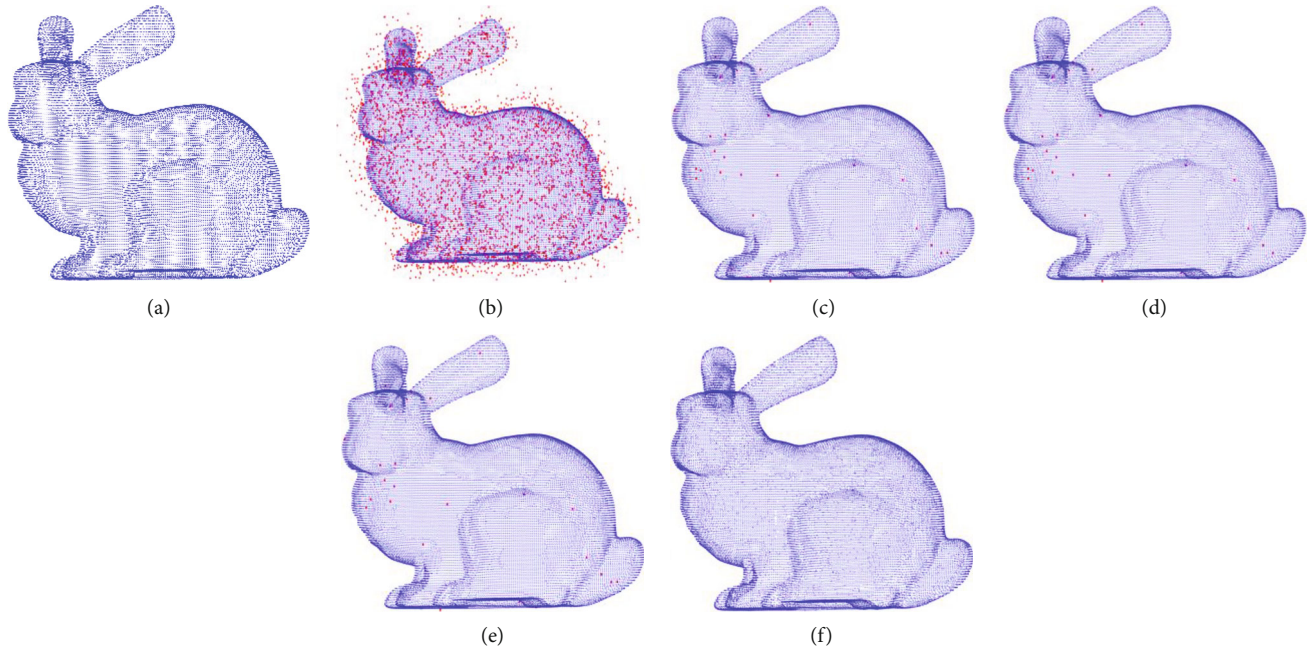


FIGURE 1: Denoising results of the three methods with 10% noise in the Bunny point cloud. (a) The original Bunny point cloud. (b) Bunny point cloud with 10% noise. (c) Statistical filtering algorithm. (d) Radius filtering algorithm. (e) Our outlier removal algorithm. (f) Our confounding point removal algorithm.

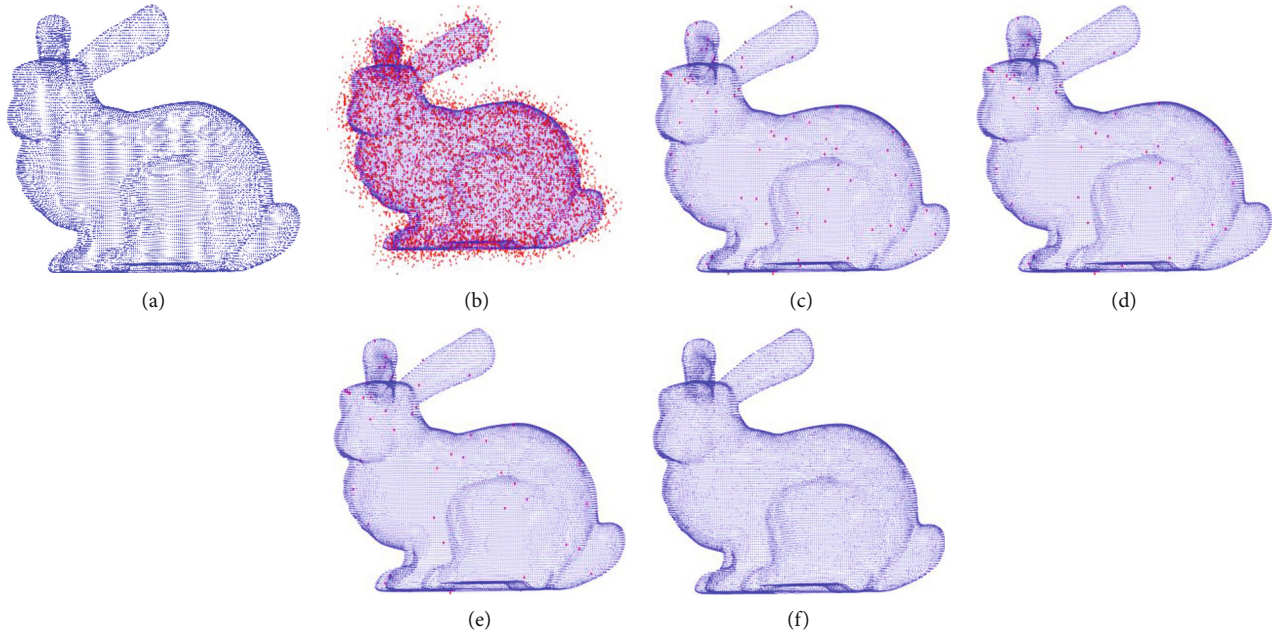


FIGURE 2: Denoising results of the three methods with 20% noise in the Bunny point cloud. (a) The original Bunny point cloud. (b) Bunny point cloud with 20% noise. (c) Statistical filtering algorithm. (d) Radius filtering algorithm. (e) Our outlier removal algorithm. (f) Our confounding point removal algorithm.

$$l = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{pmatrix}. \quad (4)$$

When d takes the minimum value, the fitting degree is the best. According to the least squares criterion, we can use the following formula to obtain B .

$$B = (U^T U)^{-1} U^T l. \quad (5)$$

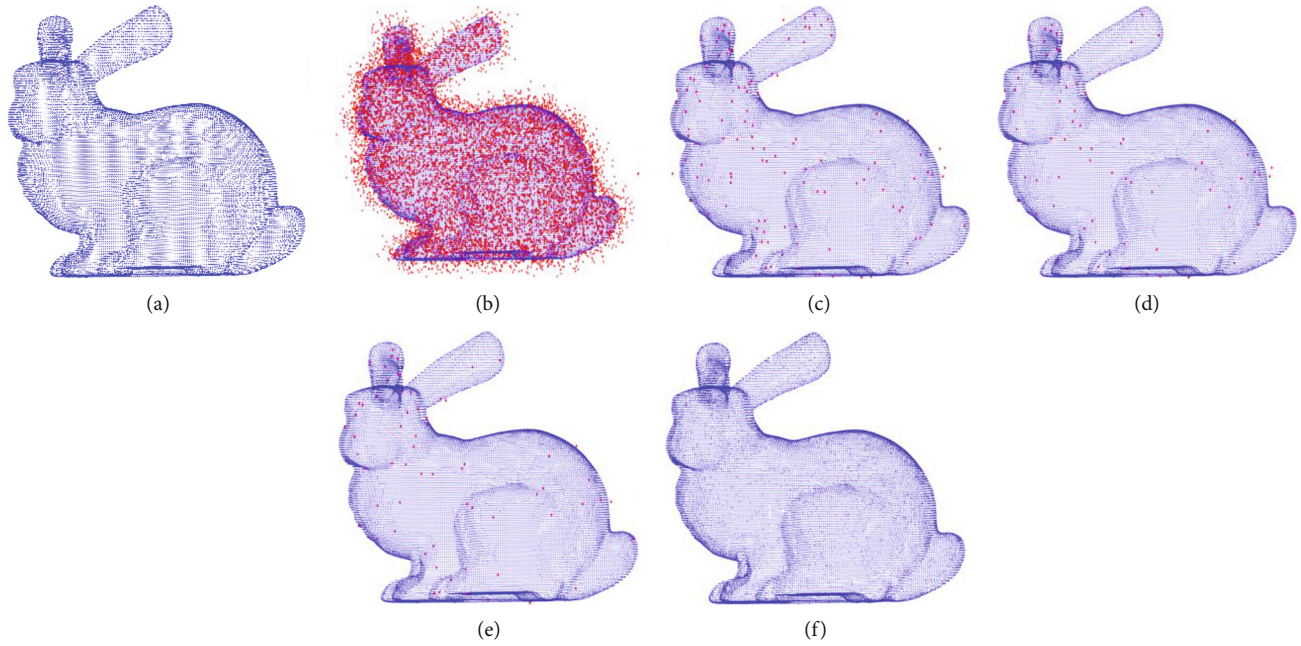


FIGURE 3: Denoising results of the three methods with 30% noise in the Bunny point cloud. (a) The original Bunny point cloud. (b) Bunny point cloud with 30% noise. (c) Statistical filtering algorithm. (d) Radius filtering algorithm. (e) Our outlier removal algorithm. (f) Our confounding point removal algorithm.

TABLE 1: Denoising results of the three methods with 10% noise in Bunny point cloud.

Algorithms	Total number of noisy point clouds	Pd	Rd	F1
Statistical filtering	39542	0.6935	0.9704	0.8091
Radius filtering	39542	0.7029	0.9814	0.8189
Our method	39542	0.7541	0.7177	0.8279

TABLE 2: Denoising results of the three methods with 20% noise in Bunny point cloud.

Algorithms	Total number of noisy point clouds	Pd	Rd	F1
Statistical filtering	43173	0.6315	0.9339	0.7535
Radius filtering	43173	0.7001	0.9848	0.8184
Our method	43173	0.7441	0.9453	0.8327

TABLE 3: Denoising results of the three methods with 30% noise in Bunny point cloud.

Algorithms	Total number of noisy point clouds	Pd	Rd	F1
Statistical filtering	46732	0.5929	0.9046	0.7163
Radius filtering	46732	0.7124	0.9646	0.8192
Our method	46732	0.7588	0.9368	0.8385

In this way, the fitting plane can be obtained, and the normal vector of a certain point can also be got by substituting the coordinates of the point into the fitting plane.

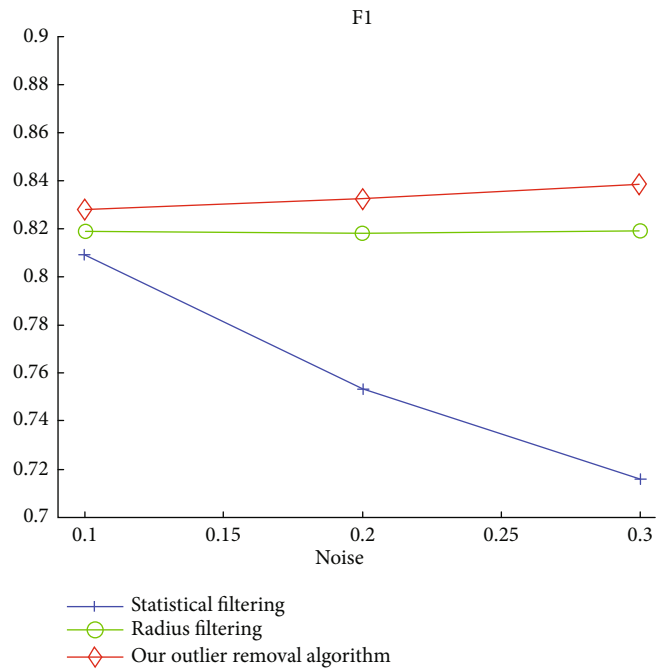


FIGURE 4: *F*-score of the three algorithms.

TABLE 4: Chamfer distance in Bunny point cloud.

Noise level	$d_{CD}(s_0, s_1)$	$d_{CD}(s_0, s_2)$	$d_{CD}(s_0, s_3)$
10%	1.5892	0.0464	0.0424
20%	2.7208	0.0748	0.0718
30%	3.7317	0.1132	0.0891

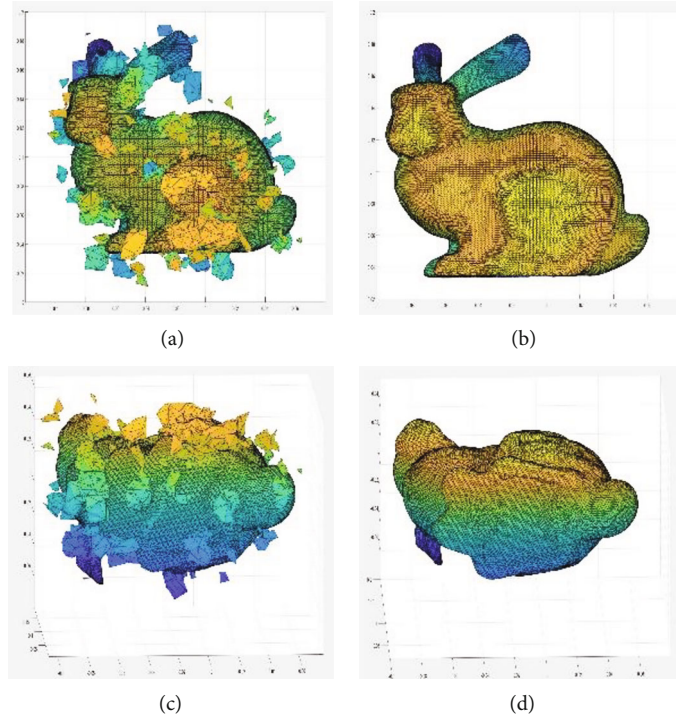


FIGURE 5: Triangulation effect. (a) Noisy point cloud triangulation effect. (b) Point cloud triangulation effect after the viewing angle is flipped. (c) Triangulation effect of the point cloud processed by the proposed algorithm after flipping the viewing angle. (d) Triangulation effect of the point cloud processed by the proposed algorithm after flipping the viewing angle.

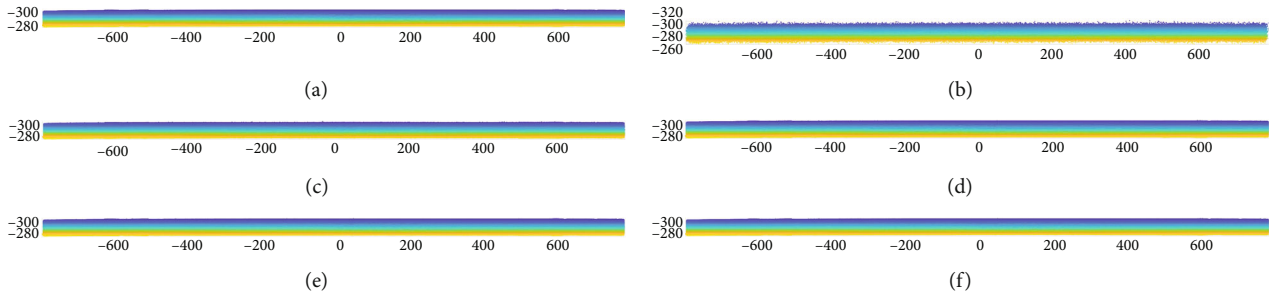


FIGURE 6: Denoising results of the three methods in the pantograph model. (a) The original pantograph point cloud. (b) Pantograph point cloud with 10% noise. (c) Statistical filtering algorithm. (d) Radius filtering algorithm. (e) Our outlier removal algorithm. (f) Our confounding point removal algorithm.

The specific implementation steps are as follows:

- (1) For a point $p_i \in P$ in the target point cloud P , searching for its k -neighborhood points and calculating the mean value p_{mi} of the neighborhood points
- (2) Using the least squares method to fit a local plane L_i and calculating the normal vector n_i of p_{mi}
- (3) Calculating the projection of the vector $p_i p_{mi}$ in the normal direction to obtain the distance d from the point $p_i \in P$ to the local fitting plane L_i

$$d = |p_i - p_m| \cdot \cos \theta \quad (6)$$

- (4) Projecting $p_i \in P$ onto the local fitting plane L_i and obtaining a projection point p'_i

$$p'_i = p_i - d \cdot n_i \quad (7)$$

- (5) Getting the new point set P'

2.3. Hole Filling. For the hole repair based on scattered point clouds, the scattered point cloud data can be triangulated first, and the possible holes in the point cloud data can be transformed into grid holes. When using the triangulation algorithm to fit the surface of a point cloud, the part of the triangle with a hole is usually relatively large, so in the process of repairing the point cloud hole, we can judge whether